



WebScraper

Parallele Programmierung – C# vs. Go

Konzepte der Programmiersprachen

Simon Liebers

21.01.2026

Ziel

WebScraper

- Einlesen einer Liste von URLs aus Datei
- Verarbeiten der Webseiten (Fetching + Parsing)
- Speicherung der Ergebnisse (Titel, Links, Bilder, Fehler) -> JSON

Config

- Input File
- Result Directory
- Degree of Parallelism (DoP)
- HTTP-Timeout
- User Agent

»» Vergleich: Parallele Programmierung

Goroutines und Channels

- Goroutines: Ausführungseinheiten
 - Eigene Goroutine für jede URL
 - Verarbeitung sofort nebenläufig
- **fetch** läuft für alle URLs parallel
- Channels: Kommunikation zwischen Goroutines
- Empfang im Hauptprogramm synchronisiert
 - Warten, bis alle Ergebnisse da sind
 - Reihenfolge der Ergebnisse nicht garantiert

🔍 Parallelität limitieren?

```
urls := []string{...}

results := make(chan string)

for _, url := range urls {
    go func(u string) {
        result := fetch(u)
        results <- result
    }(url)
}

for i := 0; i < len(urls); i++ {
    fmt.Println(<-results)
}
```

Worker Pool

- Feste Anzahl Worker
 - Degree of Parallelism
 - Keine Goroutine pro URL
- Channels
 - Aufgaben: jobs
 - Ergebnisse: results
- Kontrollierte Parallelität

```
urls := []string{...}

jobs := make(chan string, len(urls))
results := make(chan string, len(urls))

worker := func(id int, jobs <-chan string, results chan<- string) {
    for url := range jobs {
        results <- fetch(url)
    }
}

concurrency := 3 // DoP
for w := 1; w <= concurrency; w++ {
    go worker(w, jobs, results)
}

for _, url := range urls {
    jobs <- url
}
close(jobs)

for i := 0; i < len(urls); i++ {
    fmt.Println(<-results)
}
```



Tasks und WhenAll

- Tasks:
 - Nebenläufige Operationen
 - Können parallel ausgeführt werden
 - Liefert ein Ergebnis (Task<T>)
- Task.WhenAll:
 - Wartet auf alle Tasks
 - Sammelt Ergebnisse in Liste

? Parallelität limitieren?

```
var urls = new[] {...};

var tasks = urls.Select(async url =>
{
    return await FetchAsync(url);
});

var results = await Task.WhenAll(tasks);

foreach (var result in results)
{
    Console.WriteLine(result);
}
```



Semaphore

```
var urls = new[] {...};

var semaphore = new SemaphoreSlim(concurrency);
var tasks = urls.Select(async url =>
{
    await semaphore.WaitAsync();
    try
    {
        return await FetchAsync(url);
    }
    finally
    {
        semaphore.Release();
    }
});

var results = await Task.WhenAll(tasks);

foreach (var r in results)
{
    Console.WriteLine(r);
}
```



Task Parallel Library (TPL)

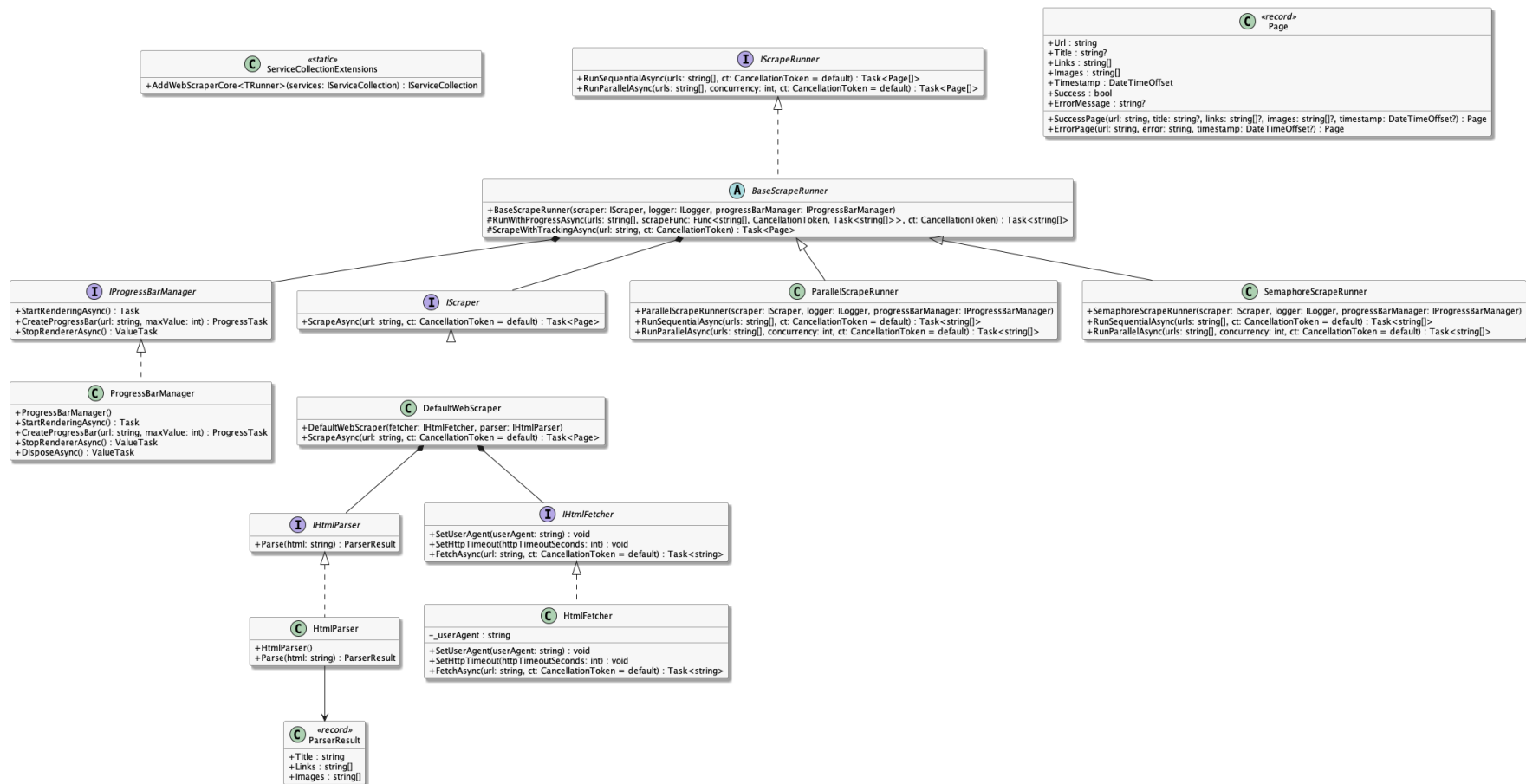
```
var results = new ConcurrentBag<string>();

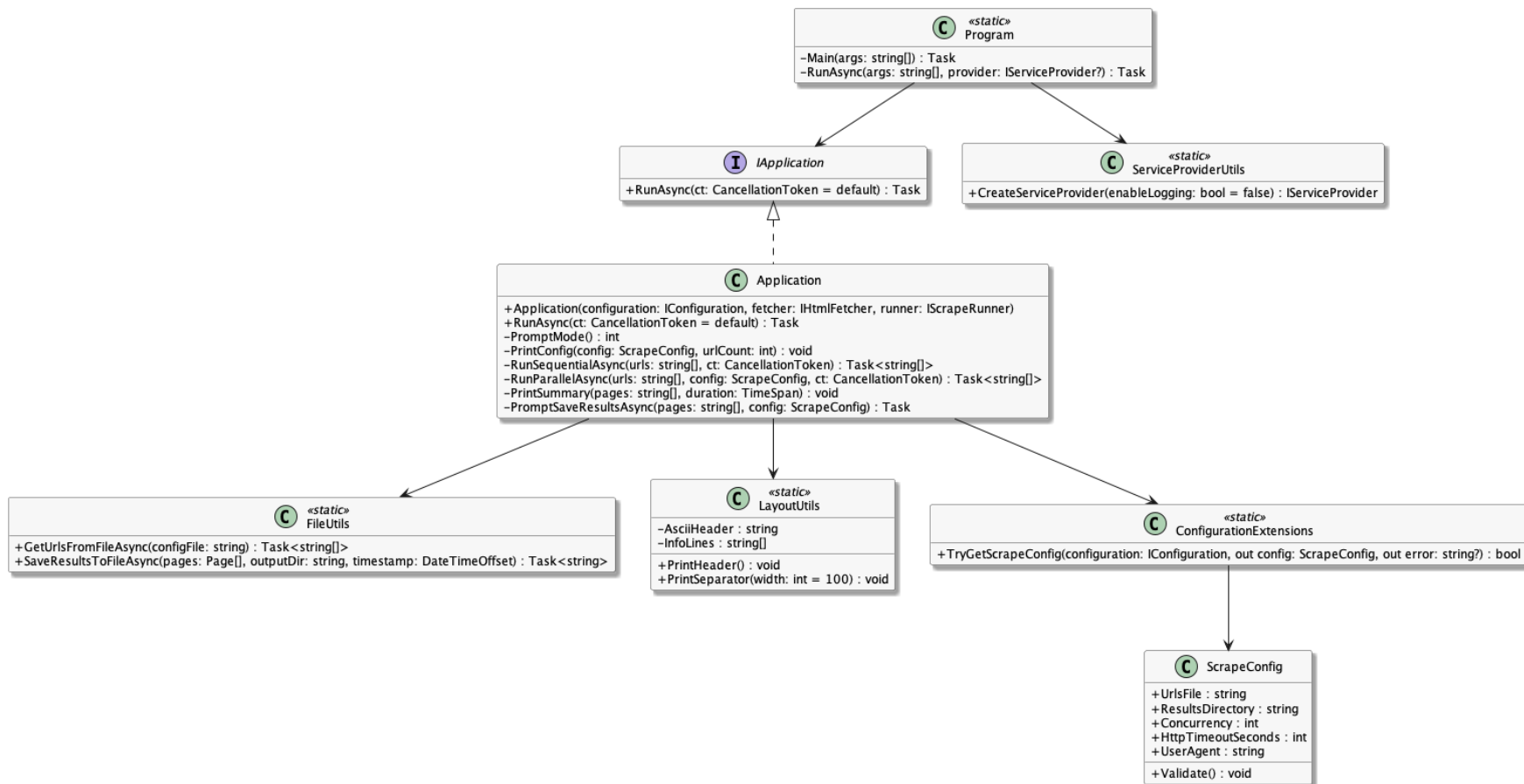
await Parallel.ForEachAsync(
    targets,
    new ParallelOptions { MaxDegreeOfParallelism = concurrency },
    async (url, ct) =>
    {
        var result = await FetchAsync(url, ct);
        results.Add(result);
    });

foreach (var r in results)
{
    Console.WriteLine(r);
}
```

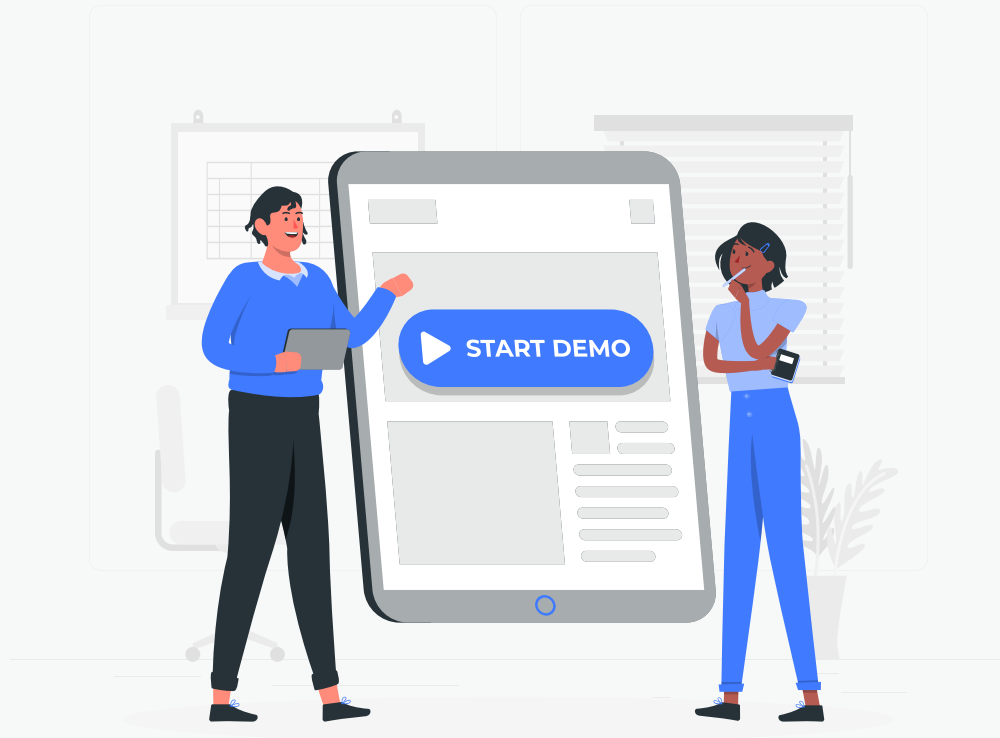
Architektur

- **WebScraper.Core:** Wiederverwendbare Bibliothek mit gesamter Logik
 - Keine UI-Logik
 - Austauschbare Parallelisierungsstrategie durch:
 - Generics
 - Dependency Injection
- **WebScraper.Cli:** Ausführbare Anwendung mit Konfiguration, Start und Darstellung
 - Verwendet Core-Library





Demo



Fazit



- Parallelität über Goroutines
- Synchronisation über Channels
- Keine eingebaute Limitierung der Parallelität
 - Worker Pools (Pattern)
- **Fokus:** Architektur und Implementierung



- Parallelität über Tasks
- Koordination über **await** und **WhenAll**, etc.
- Limitierung der Parallelität
 - TPL: MaxDegreeOfParallelism
- **Fokus:** Deklarativ konfigurieren

Danke
Noch Fragen?

